



CURSO

Bases de Datos II

Proyecto #1

Profesor

Kenneth Roberto Obando Rodriguez

Estudiantes

Danielo Wu Zhong | 2023150448

Beatriz Rebeca Díaz Gómez | 2024090972

I CICLO, 2026

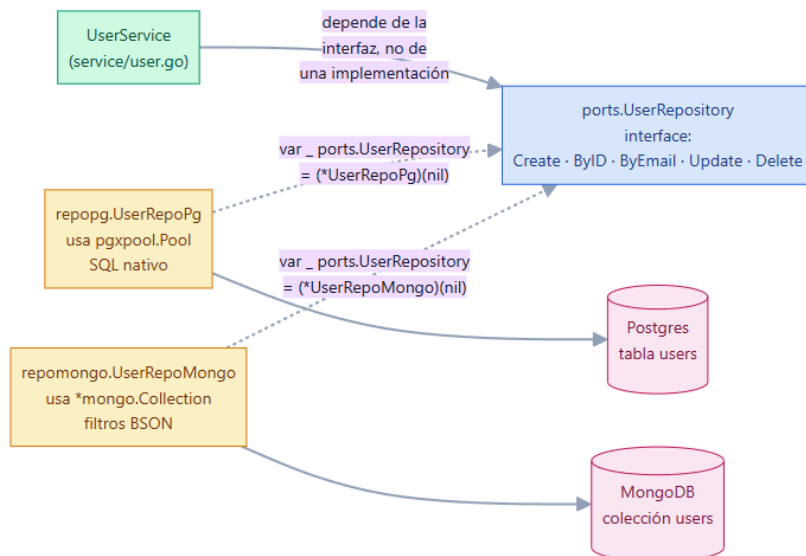
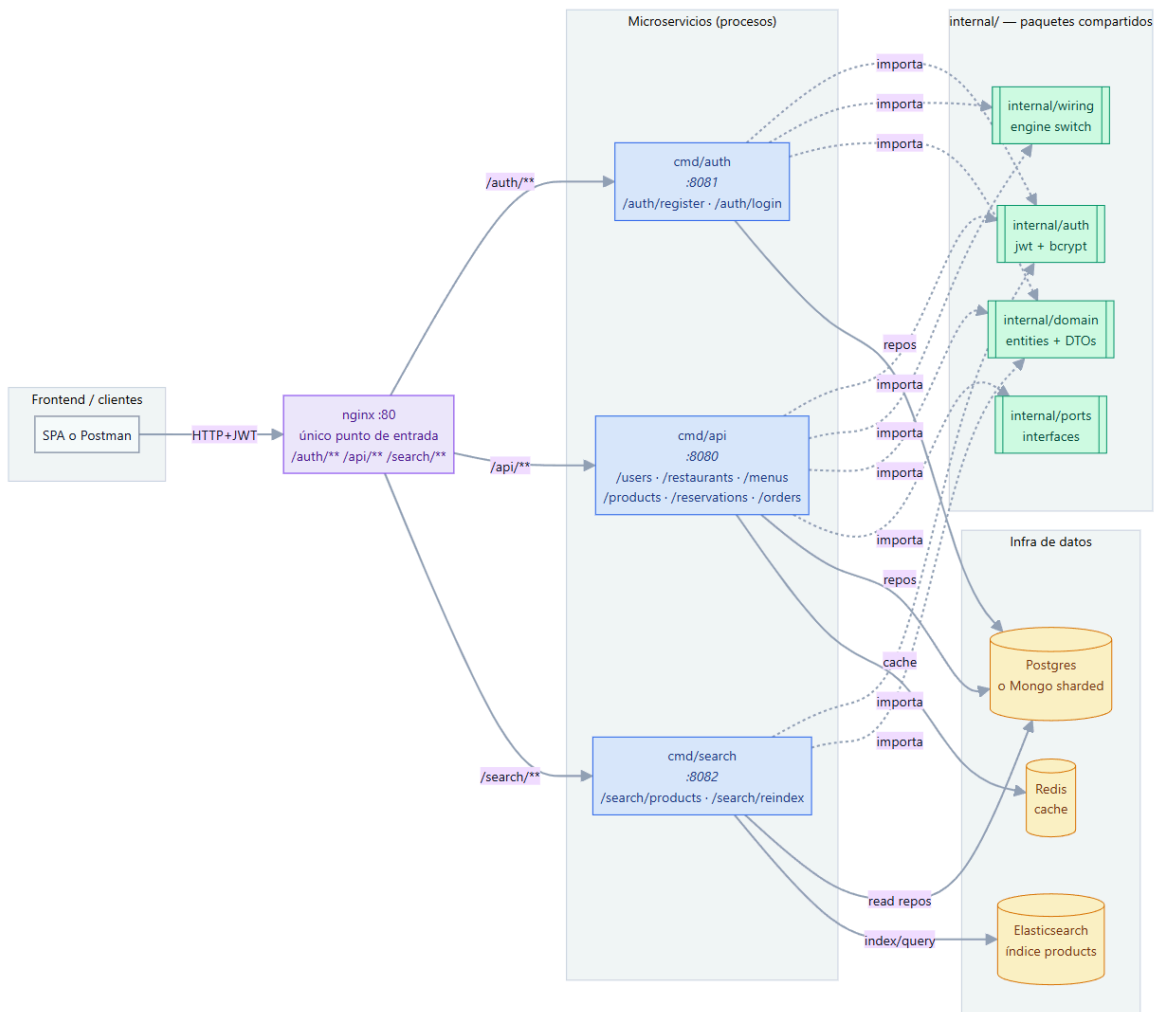
Arquitectura general del sistema	3
Decisiones de arquitectura	5
Arquitectura hexagonal (Ports and Adapters)	5
Patrón Repository (DAO)	5
Separación en microservicios	6
Shutdown ordenado	6
Cache-aside con invalidación activa	6
Modelo de datos	7
Diferencias de persistencia entre motores	8
Implementación de MongoDB	9
Implementación de Balanceador de carga y escalabilidad horizontal	11
Implementación de Elasticsearch	12
Implementación de Redis como caché	12
Pruebas unitarias implementadas	13
Pruebas unitarias en internal/transport/http	13
Pruebas unitarias en internal/adapters/repomongo	13
Pruebas unitarias en internal/adapters/searches	14
Pruebas de repopg	14
Cobertura obtenida	14
Pruebas E2E implementadas	16
Flujos E2E con PostgreSQL	16
Flujos E2E con MongoDB	16
Flujos E2E del microservicio de búsqueda	17
Generación de datos de prueba con LLM	18
Pipeline CI/CD	19
Manual de usuario	21
Requisitos previos	21
Configuración inicial	21
Levantar el sistema con PostgreSQL	21
Levantar el sistema con MongoDB	22
Generar datos de prueba	22
Flujo básico de prueba manual	24
Escalado horizontal	25
Correr las pruebas	25
Apagar el sistema	25
Conclusión	26

Arquitectura general del sistema

Se desarrolló una arquitectura de microservicios compuesta por tres servicios independientes que se comunican a través de un balanceador de carga Nginx. Cada servicio es un binario Go independiente con su propio proceso y responsabilidades bien definidas.

- **api** (cmd/api, puerto 8080) es el servicio principal. Gestiona el CRUD completo de restaurantes, menús, productos, reservaciones y órdenes. Se conecta al motor de base de datos activo (Postgres o MongoDB), a Redis para caché y expone todos los endpoints bajo /api/*.
- **auth** (cmd/auth, puerto 8081) es el servicio de autenticación. Maneja el registro de usuarios y el login, emitiendo tokens JWT firmados con un secreto compartido. Al validar tokens usa el mismo secreto que api y search, lo que hace la autenticación stateless sin llamadas HTTP entre servicios.
- **search** (cmd/search, puerto 8082) es el microservicio de búsqueda. Lee productos del repositorio principal (Postgres o MongoDB según DB_ENGINE) y los indexa en ElasticSearch. Expone endpoints de búsqueda textual, búsqueda por categoría y reindexación manual bajo /search/*.

Internamente el proyecto aplica arquitectura hexagonal (Ports and Adapters): el núcleo de negocio en internal/service/ y internal/domain/ no importa ninguna tecnología concreta. Habla únicamente con interfaces definidas en internal/ports/. Los adapters en internal/adapters/ implementan esas interfaces para cada tecnología: repopg para Postgres, repomongo para MongoDB, cacheredis para Redis y searches para ElasticSearch. El único punto donde se decide qué adapter instanciar es internal/wiring/, que lee la variable de entorno DB_ENGINE y construye el set de repositorios correspondiente. Ninguna otra parte del código contiene esa condición.



Decisiones de arquitectura

Arquitectura hexagonal (Ports and Adapters)

El principio central del diseño es que la lógica de negocio no debe conocer los detalles de infraestructura. Para lograrlo se aplicó la arquitectura hexagonal, también conocida como Ports and Adapters, propuesta por Alistair Cockburn.

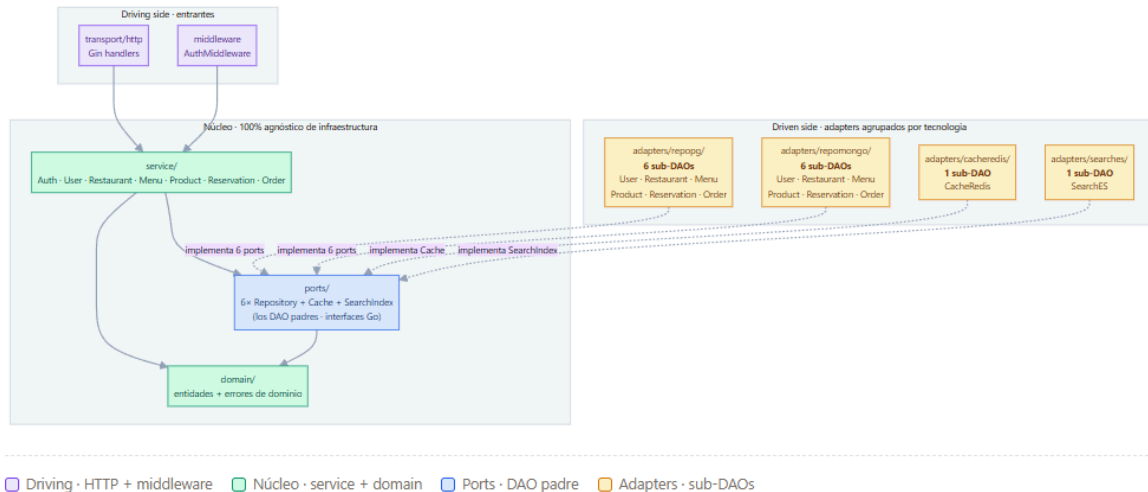
El núcleo del sistema lo forman `internal/domain/` con las entidades puras del negocio y `internal/service/` con la lógica de negocio. Ninguno de estos paquetes importa drivers de base de datos, clientes HTTP ni ninguna tecnología concreta. Toda comunicación con el exterior ocurre a través de interfaces Go definidas en `internal/ports/`, que actúan como los "puertos" del hexágono.

Los "adapters" son las implementaciones concretas de esas interfaces: `repopg` para Postgres, `repomongo` para MongoDB, `cacheredis` para Redis y `searches` para Elasticsearch. Cada adapter puede ser reemplazado sin tocar una sola línea del núcleo. Esto se demuestra en la práctica con el switch entre Postgres y MongoDB vía `DB_ENGINE`.

Patrón Repository (DAO)

Cada entidad del dominio tiene su propio repositorio definido como interfaz en `internal/ports/`: `UserRepository`, `RestaurantRepository`, `MenuRepository`, `ProductRepository`, `ReservationRepository` y `OrderRepository`. Cada uno expone únicamente los métodos que la lógica de negocio necesita, ni más ni menos.

Este patrón desacopla completamente la lógica de negocio del mecanismo de persistencia. El servicio `ReservationService`, por ejemplo, llama a `repo.CheckAvailability()` sin saber si por debajo hay una query con `EXCLUDE USING gist` en Postgres o un aggregation pipeline en MongoDB.



Separación en microservicios

La decisión de dividir en tres binarios independientes (api, auth, search) responde a responsabilidades claramente distintas y a la posibilidad de escalarlos de forma independiente. El servicio api es el más demandado y puede escalarse a N réplicas sin tocar auth ni search. El servicio auth maneja solo autenticación y tiene un perfil de carga muy diferente. El servicio search tiene su propio ciclo de vida ligado a ElasticSearch.

Los tres comparten código únicamente a través de paquetes bajo internal/, nunca mediante llamadas HTTP entre sí. El JWT firmado por auth es verificado por api y search usando el mismo JWT_SECRET, lo que mantiene la autenticación stateless sin dependencias de red entre servicios en tiempo de request.

Shutdown ordenado

Cada main.go registra un handler de SIGTERM que espera a que los requests en vuelo terminen antes de cerrar conexiones a la base de datos y salir. Esto es crítico en un entorno con múltiples réplicas donde el orquestador puede matar instancias en cualquier momento. Sin un shutdown ordenado, los requests en tránsito recibirían errores de conexión en lugar de respuestas correctas.

Cache-aside con invalidación activa

Redis no actúa como proxy transparente sino como caché controlada por la aplicación. El servicio decide explícitamente cuándo leer de caché, cuándo escribir y cuándo invalidar. Esto da control total sobre la consistencia: una escritura siempre invalida las claves afectadas en el

mismo momento, sin esperar a que expire el TTL. La búsqueda de claves a invalidar usa SCAN en lugar de KEYS para no bloquear Redis bajo carga.

Modelo de datos

El sistema maneja siete entidades definidas en `internal/domain/`. Cada struct usa tres conjuntos de tags simultáneamente: `json` para serialización HTTP, `db` para Postgres y `bson` para MongoDB. Esto permite que la misma estructura se persista en cualquiera de los dos motores sin necesidad de DTOs intermedios por adaptador.

- **User** representa una cuenta registrada en el sistema. Tiene un rol que puede ser `client` o `admin`. El campo `password` almacena el hash `bcrypt` y nunca se serializa a JSON. Los administradores son los únicos que pueden crear restaurantes y gestionar menús.
- **Restaurant** es el establecimiento. Referencia al usuario que lo administra mediante `admin_id`. Tiene nombre, dirección, teléfono, descripción y capacidad máxima.
- **Menu** pertenece a un restaurante y agrupa productos. En Postgres los productos se cargan mediante un `join` separado (el campo `Products` tiene `db:"-"`). En MongoDB los productos se embeben directamente dentro del documento del menú.
- **Product** es el ítem del menú. Tiene nombre, descripción, categoría, precio y disponibilidad. La categoría es el campo central de esta etapa: es la `shard key` hashed en MongoDB y el campo principal de indexado en ElasticSearch. Cuando un producto no tiene descripción, el método `DescriptionOrDefault()` retorna el texto *"Producto sin descripción"* para cumplir el requisito del enunciado.
- **Reservation** representa una reserva de mesa con fecha, tamaño del grupo y estado (`pending`, `confirmed`, `cancelled`). La `shard key` en MongoDB es `restaurant_id` hashed, ya que la consulta más frecuente es verificar disponibilidad por restaurante.
- **Order** es un pedido de comida. La referencia a una reservación es opcional (puntero), lo que permite pedidos sin reserva previa (`takeaway`). Al igual que los menús, en Postgres los ítems se cargan por query separada y en MongoDB se embeben en el documento.
- **OrderItem** es una línea del pedido. El precio se guarda en el momento de crear el pedido, no como referencia al producto, garantizando que si el precio del producto cambia después el pedido original conserve el valor original.

Diferencias de persistencia entre motores

En PostgreSQL las relaciones se modelan con llaves foráneas entre tablas. Los productos son filas separadas en la tabla products y se unen al menú mediante menu_id. Las órdenes tienen su propia tabla order_items relacionada por order_id.

En MongoDB los menús embeben sus productos como array dentro del documento, y las órdenes embeben sus items de la misma forma. Esto reduce los joins necesarios para las lecturas más frecuentes a costa de duplicar el precio en cada item de orden. Las colecciones products y reservations están particionadas entre los shards del cluster usando shard key hashed sobre category y restaurant_id respectivamente.

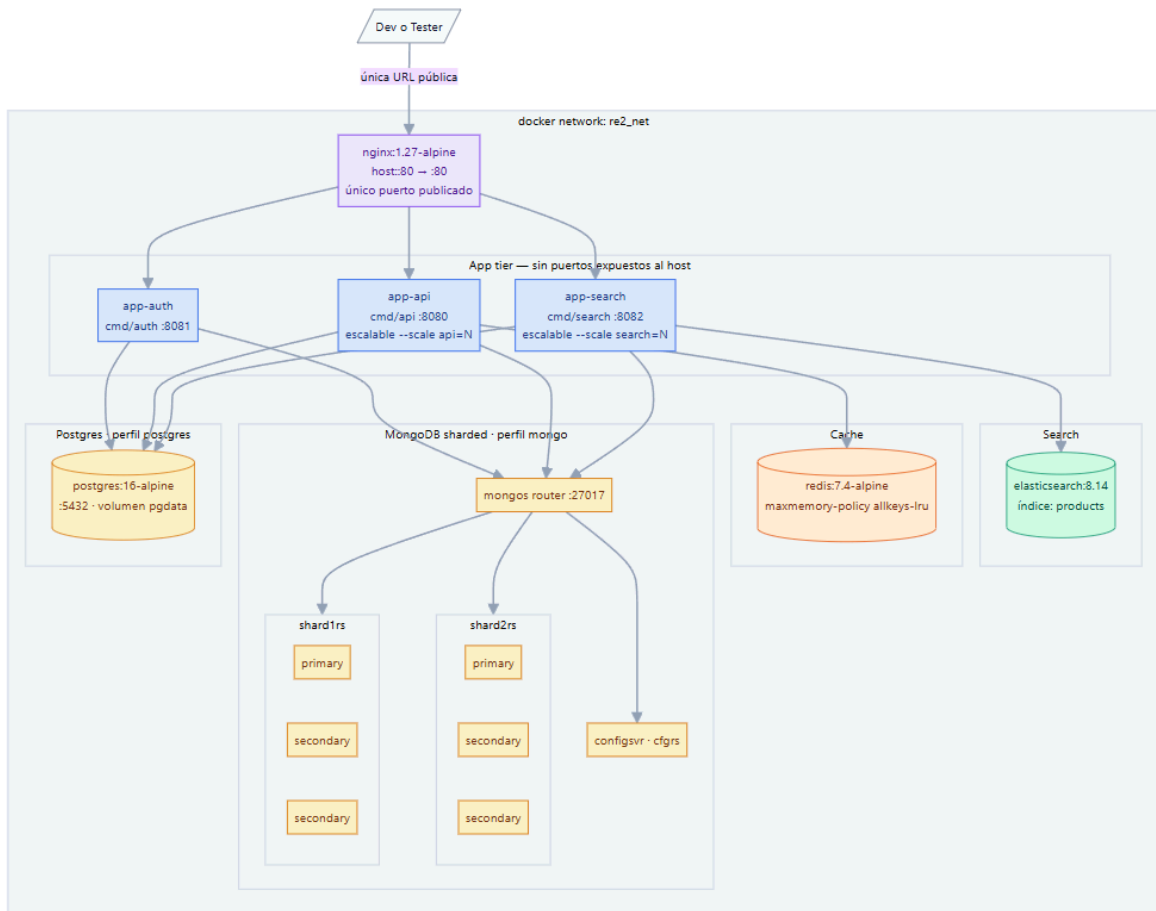
Implementación de MongoDB

Para esta etapa se integró MongoDB como motor de persistencia alternativo dentro del proyecto, permitiendo seleccionar su uso mediante configuración, sin modificar la lógica principal de negocio. Esta parte responde al requisito del proyecto de soportar compatibilidad entre PostgreSQL y MongoDB a través de patrones como Repositorio o DAO, de forma que el cambio de motor se realice por configuración y no mediante condiciones dispersas en la aplicación.

Además de la integración del motor, se validó el funcionamiento del clúster MongoDB con replicación y sharding. La configuración comprobada incluyó dos shards y, en cada shard, un conjunto de réplicas con un nodo primario y dos nodos secundarios. Esto permitió cumplir con la parte del enunciado que exige replicación operativa y particionado de datos en MongoDB.

En cuanto al particionado, se verificó que las colecciones `products` y `reservations` se distribuyeran entre los shards del clúster, cumpliendo con el requisito específico de particionar los datos de productos y reservas. La validación se realizó mediante los comandos de inspección del clúster y la revisión de la distribución efectiva de documentos entre shards, confirmando que el balanceo y la partición estaban activos y funcionando correctamente.

También se comprobó que los servicios principales del sistema pudieran ejecutarse usando MongoDB como motor de datos seleccionado. Esto se verificó al levantar los contenedores `api` y `auth` con configuración Mongo, observando que ambos servicios iniciaban correctamente y quedaban en estado saludable. Con ello se confirmó no solo la operación del clúster, sino también la integración real de MongoDB dentro de la arquitectura del sistema.



Implementación de Balanceador de carga y escalabilidad horizontal

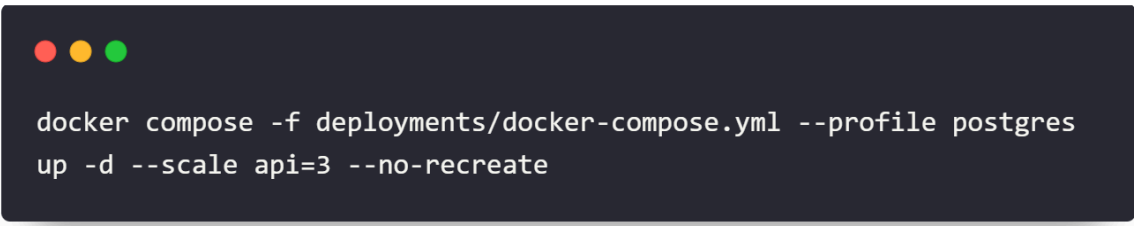
Se implementó Nginx como balanceador de carga en el puerto 80, actuando como único punto de entrada al sistema. El archivo de configuración vive en

`deployments/nginx/nginx.conf`.

El ruteo es el siguiente: las solicitudes al prefijo `/auth/*` son redirigidas al servicio `auth` en el puerto 8081, las solicitudes a `/api/*` al servicio `api` en el puerto 8080, y las solicitudes a `/search/*` al servicio `search` en el puerto 8082.

Nginx descubre las instancias activas mediante el resolver DNS interno de Docker (127.0.0.11) con re-resolución cada 5 segundos. Esto significa que al escalar un servicio, las nuevas réplicas quedan disponibles automáticamente sin reiniciar el balanceador. Si una instancia muere durante una petición, la directiva `proxy_next_upstream` reintentará la solicitud en otra instancia viva sin que el cliente perciba el error.

Los tres servicios (`api`, `auth`, `search`) están configurados en el `docker-compose.yml` con `deploy.replicas` y sin `container_name`, lo cual es el requisito de Compose para permitir el escalado horizontal. Para escalar en caliente sin bajar el stack:



```
docker compose -f deployments/docker-compose.yml --profile postgres
up -d --scale api=3 --no-recreate
```

Se validó el balanceo y el failover matando manualmente una instancia y verificando que el sistema continuará respondiendo correctamente a través de las instancias restantes.

Implementación de Elasticsearch

Se implementó un microservicio de búsqueda utilizando Elasticsearch, en concordancia con el requerimiento del proyecto de exponer endpoints para búsqueda textual, búsqueda por categoría y reindexación manual de productos. En particular, se habilitaron los endpoints:

- GET /search/products?q=texto,
- GET /search/products/category/:categoria y POST /search/reindex

Los cuales permiten consultar productos indexados y reconstruir el índice de manera controlada.

La indexación se diseñó a partir de la información de productos almacenada en MongoDB, tomando como campos principales el nombre, la categoría y la descripción. Asimismo, se contempló el caso indicado en el enunciado para productos sin descripción, asignando el valor por defecto "Producto sin descripción" cuando este campo no estuviera presente. Esta condición también fue validada durante las pruebas funcionales del microservicio.

Como parte de la validación, se comprobó que el índice products existiera en Elasticsearch, que el mapping de campos fuera coherente con el tipo de consultas requeridas y que las búsquedas devolvieran resultados correctos. Se probaron tanto búsquedas textuales como búsquedas por categoría, además de la operación de reindexado manual. Los resultados obtenidos mostraron que los productos podían ser consultados correctamente desde el índice y que el microservicio de búsqueda funcionaba de manera integrada con el resto del entorno desplegado.

Implementación de Redis como caché

Se implementó Redis como capa de caché mediante el patrón **cache-aside** en los servicios de productos, menús y restaurantes. El flujo de lectura consiste en intentar obtener el recurso desde Redis; si hay un miss, se consulta la base de datos y el resultado se almacena en caché para peticiones futuras. En operaciones de escritura o eliminación, se invalidan activamente las claves afectadas para evitar inconsistencias.

La implementación vive en `internal/adapters/cacheredis/` y expone la interfaz definida en `internal/ports/`, lo que permite que los servicios consuman la caché sin acoplarse a Redis directamente. La cobertura de este adaptador fue de 87.5%.

Recurso	Clave Redis	TTL
Producto por ID	<code>products:id:<id></code>	10 minutos
Productos por categoría	<code>products:cat:<cat></code>	10 minutos
Restaurante por ID	<code>restaurants:id:<id></code>	5 minutos
Listado de restaurantes	<code>restaurants:all</code>	5 minutos
Menú por ID	<code>menus:id:<id></code>	5 minutos

Redis opera con `maxmemory 256mb` y política de evicción `allkeys-lru`, por lo que cuando el caché alcanza su límite de memoria evicta automáticamente las claves menos recientemente usadas sin intervención manual. La invalidación activa se realiza mediante `SCAN` en lotes de 500 entradas, evitando el uso de `KEYS *` que bloquearía Redis en producción.

Pruebas unitarias implementadas

Pruebas unitarias en `internal/transport/http`

En el paquete `internal/transport/http` se desarrollaron pruebas unitarias enfocadas en la capa de transporte HTTP, validando el comportamiento de handlers y middleware del sistema. En esta sección se incluyeron pruebas para autenticación, restaurantes, menús, productos, usuarios, reservaciones y órdenes, utilizando repositorios simulados para aislar la lógica del handler y comprobar respuestas HTTP, códigos de estado y validación de entradas.

Además de los caminos exitosos, estas pruebas cubrieron casos de error y control de acceso, incluyendo solicitudes inválidas, acceso no autorizado, rutas protegidas sin token, validación de roles administrativos y manejo centralizado de errores. También se probaron componentes transversales como el router, el middleware de autenticación, el middleware de administrador y el handler del microservicio de búsqueda. Como resultado, este paquete alcanzó una cobertura de **88.6%**, lo que refleja un nivel alto de validación para la capa HTTP.

Pruebas unitarias en `internal/adapters/repomongo`

En `internal/adapters/repomongo` se implementaron pruebas unitarias para los sub-DAO de MongoDB correspondientes a usuarios, restaurantes, menús, productos, reservaciones y órdenes. Estas pruebas validaron operaciones de creación, consulta, actualización y eliminación, así como comportamientos ante datos inexistentes y generación automática de identificadores y fechas cuando correspondía.

También se contemplaron flujos específicos del acceso a datos en MongoDB, por ejemplo la actualización de menús con reemplazo de productos asociados, la verificación de reservaciones, la cancelación de registros y la obtención de documentos por identificador. En conjunto, estas pruebas permitieron comprobar la consistencia del acceso a datos cuando el sistema opera sobre MongoDB. La cobertura alcanzada por este paquete fue de **80.8%**.

Pruebas unitarias en `internal/adapters/searches`

En el paquete `internal/adapters/searches` se desarrollaron pruebas unitarias para la integración lógica con Elasticsearch. En esta parte se validaron la creación del índice, la indexación individual y masiva de productos, la búsqueda textual, la búsqueda por categoría y la eliminación de documentos del índice.

Asimismo, se incluyeron escenarios de error y validaciones adicionales, tales como respuestas incorrectas del servidor, productos sin identificador, manejo de productos sin descripción y construcción correcta del cliente de Elasticsearch. Estas pruebas permitieron comprobar la lógica interna del adaptador de búsqueda sin depender exclusivamente de la ejecución E2E. La cobertura obtenida en este paquete fue de **87.8%**.

Pruebas de repopg

En `internal/adapters/repopg` se implementaron pruebas de integración para los sub-DAO de PostgreSQL, validando operaciones de creación, consulta, actualización y eliminación para usuarios, restaurantes, menús, productos, reservaciones y órdenes contra una instancia real de Postgres. Estas pruebas complementan las de repomongo y confirman que ambos adapters cumplen el mismo contrato definido por los ports, garantizando la intercambiabilidad entre motores.

Cobertura obtenida

A continuación se presenta la cobertura por paquete obtenida al ejecutar la suite de pruebas completa (unitarias + integración) contra Postgres y MongoDB:

Paquete	Cobertura
internal/domain	100.0%
internal/service/product	100.0%
internal/service/restaurant	100.0%
internal/service/user	100.0%
internal/service/menu	97.5%
internal/service/order	97.1%
internal/service/reservation	91.7%
internal/service/auth	90.0%
internal/auth/jwt	92.3%
internal/auth/password	80.0%
internal/config	95.7%
internal/transport/http	88.6%
internal/adapters/searches	87.8%
internal/adapters/cachedredis	87.5%
internal/adapters/repomongo	80.8%

```

✓ Run tests

1 ▶ Run go test ./internal/... \
14 ok   restaurants-e2/internal/adapters/cachedredis    0.089s coverage: 96.9% of statements
15 ok   restaurants-e2/internal/adapters/repomongo      0.515s coverage: 82.6% of statements
16 ok   restaurants-e2/internal/adapters/repopg 0.721s coverage: 90.9% of statements
17 ok   restaurants-e2/internal/adapters/searches      0.015s coverage: 87.8% of statements
18     restaurants-e2/internal/ports                coverage: 0.0% of statements
19 ok   restaurants-e2/internal/auth    1.967s coverage: 88.9% of statements
20 ok   restaurants-e2/internal/config  0.003s coverage: 95.7% of statements
21 ok   restaurants-e2/internal/domain  0.003s coverage: 100.0% of statements
22 ok   restaurants-e2/internal/e2e     4.359s coverage: [no statements]
23 ok   restaurants-e2/internal/service 4.021s coverage: 96.4% of statements
24 ok   restaurants-e2/internal/transport/http 0.633s coverage: 91.9% of statements

✓ Check coverage ≥ 90%

1 ▶ Run go tool cover -func=coverage.out | tail -1
11 total: (statements) 90.1%
12 Cobertura total: 90.1%
```

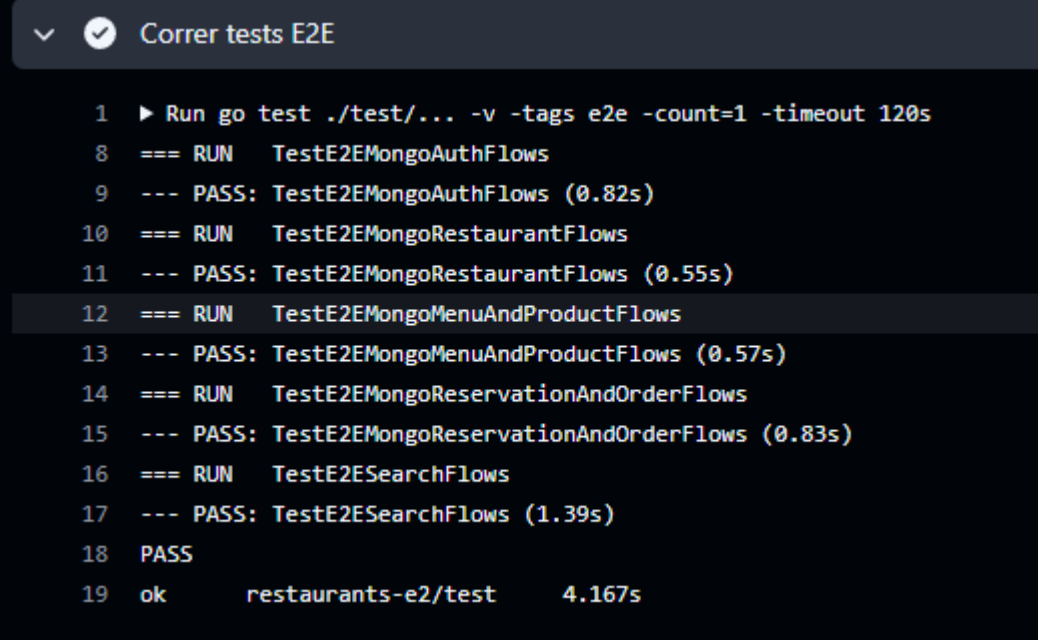
Pruebas E2E implementadas

Flujos E2E con PostgreSQL

Adicionalmente a los flujos E2E con MongoDB, se implementaron pruebas de extremo a extremo que validan el sistema usando PostgreSQL como motor de persistencia. Estas pruebas viven en `internal/e2e/` y se ejecutan contra Postgres directamente (sin requerir el stack completo de Docker Compose), lo que las hace más rápidas de correr en entornos de CI.

Los flujos cubiertos incluyen autenticación (registro y login), gestión de restaurantes, creación y actualización de menús con productos, reservaciones y órdenes. Al igual que los flujos de MongoDB, se validaron tanto los caminos exitosos como los casos de error: credenciales incorrectas, acceso sin token, operaciones administrativas por usuarios no autorizados y consultas sobre recursos inexistentes.

La existencia de pruebas E2E para ambos motores garantiza que la capa de repositorio funcione correctamente independientemente del motor configurado, validando en la práctica la intercambiabilidad que ofrece el patrón Repositorio.

A terminal window with a dark background and light-colored text. At the top, there is a header bar with a downward arrow, a checkmark icon, and the text "Correr tests E2E". Below this, the terminal shows a series of commands and their outputs. The first line is a command to run tests with specific flags. Subsequent lines show the execution of various test suites, each preceded by "=== RUN" and followed by "PASS" and a duration in parentheses. The final line shows the overall result "PASS" and a total duration.

```
1  ▶ Run go test ./test/... -v -tags e2e -count=1 -timeout 120s
8  === RUN    TestE2EMongoAuthFlows
9  --- PASS:  TestE2EMongoAuthFlows (0.82s)
10 === RUN    TestE2EMongoRestaurantFlows
11 --- PASS:  TestE2EMongoRestaurantFlows (0.55s)
12 === RUN    TestE2EMongoMenuAndProductFlows
13 --- PASS:  TestE2EMongoMenuAndProductFlows (0.57s)
14 === RUN    TestE2EMongoReservationAndOrderFlows
15 --- PASS:  TestE2EMongoReservationAndOrderFlows (0.83s)
16 === RUN    TestE2ESearchFlows
17 --- PASS:  TestE2ESearchFlows (1.39s)
18 PASS
19 ok        restaurants-e2/test    4.167s
```

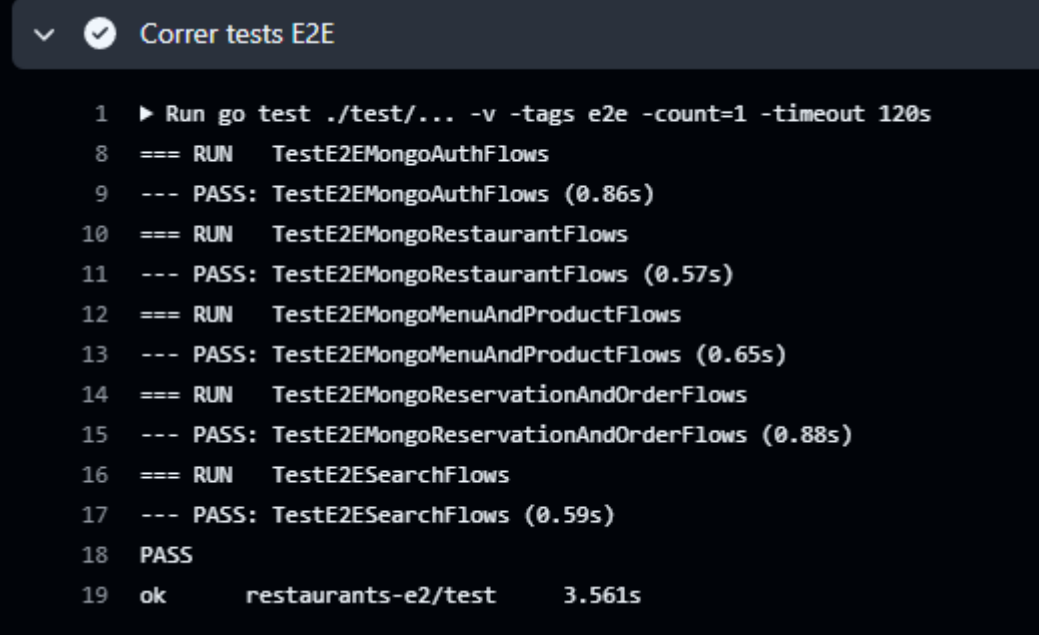
Flujos E2E con MongoDB

Además de las pruebas unitarias, se implementaron pruebas de extremo a extremo utilizando el entorno real desplegado con Docker Compose y MongoDB como motor de persistencia.

Estas pruebas permitieron validar flujos completos a través de Nginx, auth, api y la base de datos MongoDB, garantizando que los componentes trabajaran de forma integrada.

Dentro de los flujos E2E validados se incluyeron procesos de autenticación, creación y consulta de restaurantes, creación, actualización y eliminación de menús, obtención y modificación de productos, creación y cancelación de reservaciones y creación y consulta de órdenes. También se contemplaron casos de error, como credenciales incorrectas, acceso a rutas protegidas sin token, intentos de operaciones administrativas por usuarios no autorizados y consultas sobre recursos inexistentes.

La ejecución final de la suite E2E mostró resultados satisfactorios en los flujos TestE2EMongoAuthFlows, TestE2EMongoRestaurantFlows, TestE2EMongoMenuAndProductFlows y TestE2EMongoReservationAndOrderFlows, confirmando que la integración entre servicios y MongoDB se encontraba funcionando correctamente.



```
1  ▶ Run go test ./test/... -v -tags e2e -count=1 -timeout 120s
8  === RUN    TestE2EMongoAuthFlows
9  --- PASS: TestE2EMongoAuthFlows (0.86s)
10 === RUN    TestE2EMongoRestaurantFlows
11 --- PASS: TestE2EMongoRestaurantFlows (0.57s)
12 === RUN    TestE2EMongoMenuAndProductFlows
13 --- PASS: TestE2EMongoMenuAndProductFlows (0.65s)
14 === RUN    TestE2EMongoReservationAndOrderFlows
15 --- PASS: TestE2EMongoReservationAndOrderFlows (0.88s)
16 === RUN    TestE2ESearchFlows
17 --- PASS: TestE2ESearchFlows (0.59s)
18 PASS
19 ok        restaurants-e2/test    3.561s
```

Flujos E2E del microservicio de búsqueda

También se implementó una prueba E2E específica para el microservicio de búsqueda, validando el funcionamiento real del servicio search a través del balanceador. En esta suite se comprobó el reindexado manual de productos, la búsqueda textual, la búsqueda por categoría y el manejo del caso en que falta el parámetro **q** en la consulta.

Como parte importante de esta validación, se confirmó que los productos sin descripción fueran indexados utilizando el texto por defecto "Producto sin descripción" como lo exige el proyecto. La ejecución final de TestE2ESearchFlows fue satisfactoria, por lo que se consideró comprobado el funcionamiento del microservicio de búsqueda en un escenario de integración real.

Generación de datos de prueba con LLM

Para la generación de datos de prueba realistas se implementó un script en scripts/seed/ que utiliza la API de Anthropic como LLM para generar restaurantes, menús, productos y usuarios con nombres, descripciones y precios coherentes con el contexto costarricense.

El script acepta parámetros configurables de volumen (-restaurants, -menus-per, -products-per, -users) y un modo -dry-run que genera los datos sin insertarlos en la base de datos. El seeder se conecta directamente al motor de base de datos activo usando la misma configuración que los servicios.

Para garantizar que el sistema sea demostrable sin necesidad de una API key de Anthropic, se implementó un cliente estático (StaticClient) que actúa como fallback automático cuando la variable de entorno ANTHROPIC_API_KEY no está presente. Este cliente implementa la misma interfaz LLMClient que el cliente real, por lo que el resto del seed no requiere ningún cambio. Los datos estáticos incluyen diez restaurantes costarricenses con nombres, direcciones, teléfonos y descripciones realistas, veinte usuarios con nombres y correos típicos del país, y dos menús con ocho platos auténticos cada uno (casado, gallo pinto, ceviche de corvina, arroz con camarones, entre otros).



```
ANTHROPIC_API_KEY=sk-...

# Mongo (Docker corriendo)
DB_ENGINE=mongo go run ./scripts/seed/... --mongo-uri mongodbd://localhost:27017

# Postgres (Docker corriendo)
DB_ENGINE=postgres go run ./scripts/seed/... --pg-dsn postgres://postgres:postgres@localhost:5432/restaurants
```

Pipeline CI/CD

Se configuró un pipeline de integración y entrega continua en GitHub Actions que se ejecuta en cada push a las ramas tests, dev y main. El archivo de configuración vive en `.github/workflows/ci.yml`.

Los jobs definidos y su lógica de ejecución son los siguientes:

lint - Corre en todos los branches. Ejecuta `go mod verify`, `go vet ./...` y `staticcheck` sobre todo el árbol de paquetes para garantizar que el código compile y no tenga problemas estáticos.

test - Corre únicamente en el branch tests. Levanta servicios de Postgres 16 y MongoDB 7 como contenedores de GitHub Actions, aplica el schema de Postgres y ejecuta `go test ./internal/...` con cobertura atómica. El job falla si la cobertura total es menor al 90%.

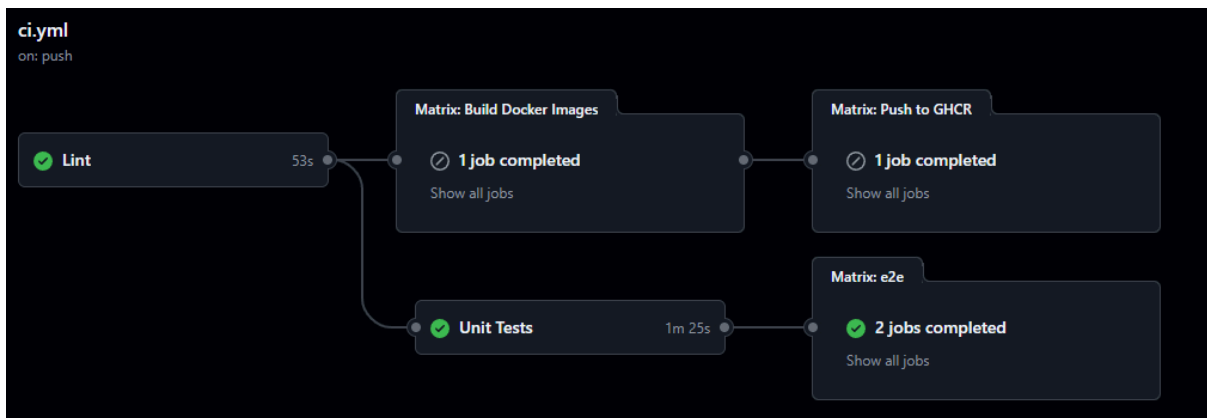
e2e - Corre en el branch tests usando una matrix strategy con dos configuraciones en paralelo: una con `DB_ENGINE=postgres` y otra con `DB_ENGINE=mongo`. Cada configuración levanta el stack completo con Docker Compose, espera que todos los servicios estén healthy y ejecuta `go test ./test/... -tags e2e`.

build - Corre en dev y main. Construye las imágenes Docker de los tres servicios (api, auth, search) en paralelo usando matrix strategy con caché de capas por GitHub Actions Cache.

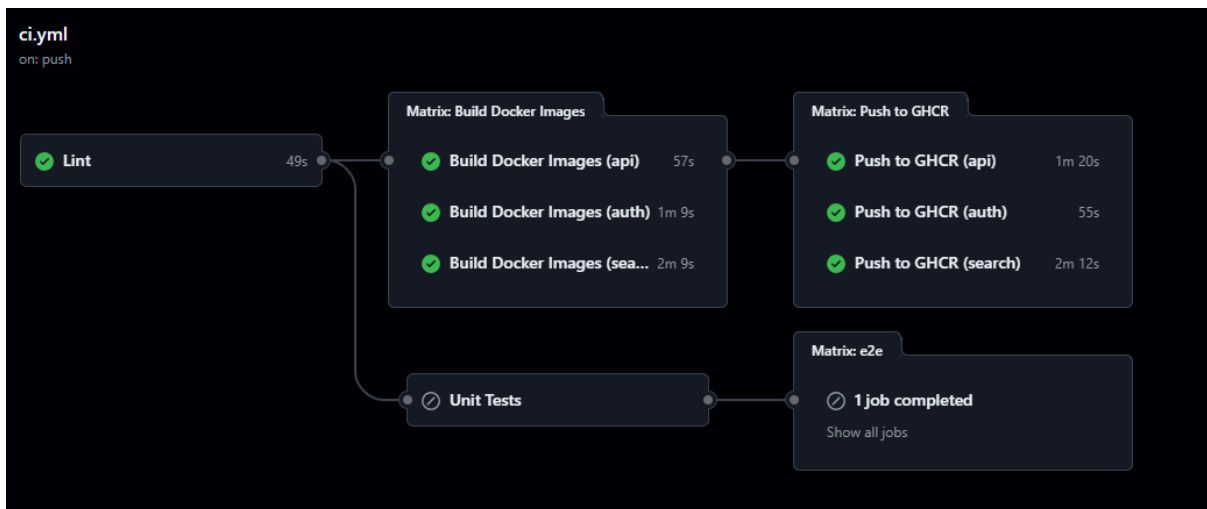
push - Corre únicamente en push a main. Publica las tres imágenes en el GitHub Container Registry (`ghcr.io/<owner>/<repo>/<servicio>`) con dos tags: latest y el SHA del commit.

Branch	Jobs que corren
tests	lint → test (cobertura $\geq$ 90%) → e2e (Postgres + Mongo en paralelo)
dev	lint → build de las 3 imágenes
main	lint → build → push a GHCR

Tests workflow



Main workflow



Manual de usuario

Requisitos previos

Para ejecutar el sistema se necesita tener instalado Docker 24 o superior con Docker Compose v2, Go 1.23 o superior (únicamente si se desea correr pruebas o el seeder fuera de contenedor), y Make (opcional, como atajo a los comandos más comunes).

Configuración inicial

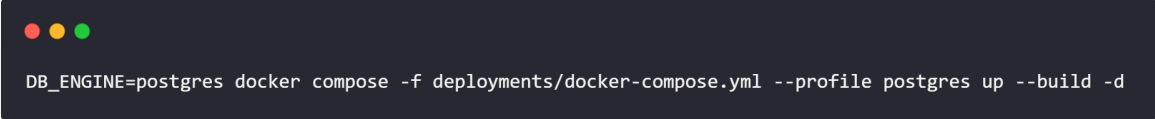
Antes de levantar el stack por primera vez se debe copiar el archivo de variables de entorno y ajustar los valores necesarios:



```
cp .env.example .env
```

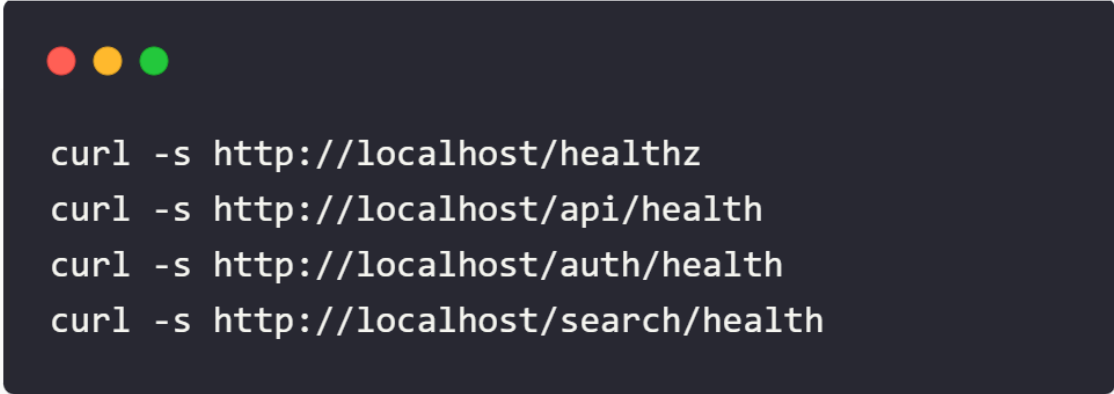
Los valores que se deben revisar son `DB_ENGINE` (acepta postgres o mongo), `JWT_SECRET` (se recomienda un valor aleatorio de 32 caracteres o más) y `ANTHROPIC_API_KEY` (opcional, solo necesaria si se quiere usar el seeder con datos generados por LLM).

Levantar el sistema con PostgreSQL



```
DB_ENGINE=postgres docker compose -f deployments/docker-compose.yml --profile postgres up --build -d
```

Para verificar que todos los servicios están saludables:



```
curl -s http://localhost/healthz  
curl -s http://localhost/api/health  
curl -s http://localhost/auth/health  
curl -s http://localhost/search/health
```

Levantar el sistema con MongoDB

```
DB_ENGINE=mongo docker compose -f deployments/docker-compose.yml --profile mongo up --build -d
```

El stack de MongoDB tarda entre 60 y 90 segundos en estar listo porque el contenedor `mongo_init` inicializa el cluster de forma automática: configura los replica sets, agrega los shards y habilita el sharding en las colecciones `products` y `reservations`. Para verificar que el cluster quedó correctamente configurado:

```
docker exec -it re2_mongos mongosh --eval 'sh.status()'
```

Generar datos de prueba

Una vez que el stack está saludable se puede poblar la base de datos con el seeder:

```
ANTHROPIC_API_KEY=sk-...

# Mongo (Docker corriendo)
DB_ENGINE=mongo go run ./scripts/seed/... --mongo-uri mongodb://localhost:27017

# Postgres (Docker corriendo)
DB_ENGINE=postgres go run ./scripts/seed/... --pg-dsn postgres://postgres:postgres@localhost:5432/restaurants
```

Si la variable `ANTHROPIC_API_KEY` está configurada en el `.env`, el seeder usará el LLM para generar datos realistas. Si no está presente, utilizará automáticamente el conjunto de datos estáticos con restaurantes, menús y usuarios costarricenses predefinidos.

Para explorar qué se generaría sin insertar nada:



```
go run ./scripts/seed --dry-run
```

Después de insertar los datos, si los endpoints devuelven resultados vacíos, basta con limpiar el caché de Redis:



```
docker exec re2_redis redis-cli FLUSHDB
```

Flujo básico de prueba manual

```

# 1. Registrar un admin
ADMIN=$(curl -s -X POST http://localhost/auth/register \
  -H "Content-Type: application/json" \
  -d '{"name":"Admin","email":"admin@demo.com","password":"admin123","role":"admin"}')
ADMIN_TOKEN=$(echo $ADMIN | jq -r '.token')

# 2. Registrar un cliente
CLIENT=$(curl -s -X POST http://localhost/auth/register \
  -H "Content-Type: application/json" \
  -d '{"name":"Cliente","email":"cliente@demo.com","password":"client123","role":"client"}')
CLIENT_TOKEN=$(echo $CLIENT | jq -r '.token')

# 3. Crear restaurante
REST=$(curl -s -X POST http://localhost/api/restaurants \
  -H "Authorization: Bearer $ADMIN_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"La Cantina",
    "address":"Av. Central",
    "phone":"2222-3333",
    "description":"Cocina tica",
    "capacity":50}')
REST_ID=$(echo $REST | jq -r '.id')

# 4. Crear menú con productos
MENU=$(curl -s -X POST http://localhost/api/menus \
  -H "Authorization: Bearer $ADMIN_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"restaurant_id":"$REST_ID",
    "name":"Menú del día",
    "description":"Almuerzo",
    "products":[{"name":"Casado",
      "description":"Arroz y frijoles",
      "category":"plato fuerte",
      "price":4500,
      "available":true}]}')
MENU_ID=$(echo $MENU | jq -r '.id')

# 5. Crear reservación
curl -s -X POST http://localhost/api/reservations \
  -H "Authorization: Bearer $CLIENT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"restaurant_id":"$REST_ID","date":"2026-06-01T12:00:00Z","people":2}' | jq

# 6. Buscar productos
curl -s "http://localhost/search/products?q=casado" | jq

```

Escalado horizontal

Para escalar cualquiera de los servicios sin bajar el stack:

```
# Escalar api a 3 réplicas
docker compose -f deployments/docker-compose.yml --profile postgres up -d --scale api=3 --no-recreate

# Ver el estado de todos los contenedores
docker compose -f deployments/docker-compose.yml ps

# Verificar que Nginx distribuye entre las réplicas
docker logs -f re2_nginx 2>&1 | grep upstream
```

Correr las pruebas

```
# Pruebas unitarias e integración (requiere Postgres en :5432 y Mongo en :27017)
POSTGRES_TEST_URL=postgres://postgres:postgres@localhost:5432/restaurants \
MONGO_TEST_URI=mongodb://localhost:27017 \
go test ./internal/... -count=1 -coverprofile=coverage.out -timeout 120s

# Ver cobertura total
go tool cover -func=coverage.out | tail -1

# Pruebas E2E (requiere el stack levantado con mongo)
go test ./test/... -v -tags e2e -count=1 -timeout 120s
```

Apagar el sistema

```
# Apagar sin borrar datos
docker compose -f deployments/docker-compose.yml --profile postgres down

# Apagar y borrar volúmenes (pierde todos los datos)
docker compose -f deployments/docker-compose.yml --profile postgres down -v
```

Conclusión

El sistema desarrollado en esta etapa cumple con la totalidad de los requisitos establecidos en el enunciado del proyecto. Se implementó una arquitectura de microservicios con tres servicios independientes (api, auth y search) detrás de un balanceador Nginx, con soporte dual para PostgreSQL y MongoDB configurable mediante variable de entorno sin modificar el código fuente. La arquitectura hexagonal aplicada garantiza que esta intercambiabilidad sea real y no superficial: el núcleo de negocio no conoce ningún detalle de infraestructura.

El cluster MongoDB opera con dos shards, cada uno con un replica set de un primario y dos secundarios, y las colecciones de productos y reservaciones están correctamente particionadas. Elasticsearch expone los tres endpoints requeridos y maneja el caso de productos sin descripción. Redis implementa el patrón cache-aside con políticas de expiración y invalidación activa. El pipeline de CI/CD verifica automáticamente que la cobertura de pruebas no baje del 90% en cada push, construye las imágenes Docker y las publica al GitHub Container Registry en cada merge a main.

El resultado es un sistema que no solo cumple los requisitos funcionales del enunciado sino que aplica patrones y prácticas del desarrollo de software profesional: separación de responsabilidades, contratos bien definidos entre capas, pruebas en tres niveles, documentación técnica detallada y un pipeline reproducible que cualquier persona puede ejecutar siguiendo las instrucciones del README.